



Machine Learning LAB

Week 02

Lab Contents

1. Linear Regression

I) Univariate Linear Regression

II) Multivariate Regression

2. Logistic Regression

3. Polynomial Regression

Univariate Linear Regression

$$\hat{y} = \beta_0 + \beta_1 x$$

Solution

These equations have a unique solution:

$$w_1 = \frac{N(\sum x_j y_j) - (\sum x_j)(\sum y_j)}{N(\sum x_j^2) - (\sum x_j)^2}; \quad w_0 = (\sum y_j - w_1(\sum x_j))/N$$

X	Y	(X ²)	(Y ²)	(X*Y)
1	2	1	4	2
2	4	4	16	8
3	5	9	25	15
4	4	16	16	16
5	5	25	25	25
Σ=15	Σ=20	Σ=55	Σ=86	Σ=66

$$w_1 = \frac{(5*66) - (15*20)}{5*55 - (15)^2} = 0.6$$

$$\hat{y} = 2.2 + 0.6x$$

$$w_0 = (20 - 0.6(15))/5 = 2.2$$

$$\hat{y} = 2.2 + 0.6(2) = 3.4$$

Load Dataset

- import pandas as pd
- `df=pd.read_csv("/content/drive/My Drive/ML_Labs/DataSets/headbrain.csv")`
- `df.head()`

	Gender	Age Range	Head Size(cm^3)	Brain Weight(grams)
0	1	1	4512	1530
1	1	1	3738	1297
2	1	1	4261	1335
3	1	1	3777	1282
4	1	1	4177	1590

Load Dataset

- # Computing X and Y
- $X = df['\text{Head Size}(\text{cm}^3)'].values$
- $y = df['\text{Brain Weight}(\text{grams})'].values$

Calculate Parameters

- `import numpy as np`
- `X_sum = np.sum(X)`
- `X_squared_sum = np.sum(X * X)`
- `n = len(X)`
`print("Sum of X:", X_sum)`
- `print("Sum of X squared:", X_squared_sum)`
- `print("Length of X:", n)`

Sum of X: 861256

Sum of X squared: 3161283190

Length of X: 237

$$w_1 = \frac{N(\sum x_j y_j) - (\sum x_j)(\sum y_j)}{N(\sum x_j^2) - (\sum x_j)^2}; \quad w_0 = (\sum y_j - w_1(\sum x_j))/N$$

Estimate Parameters

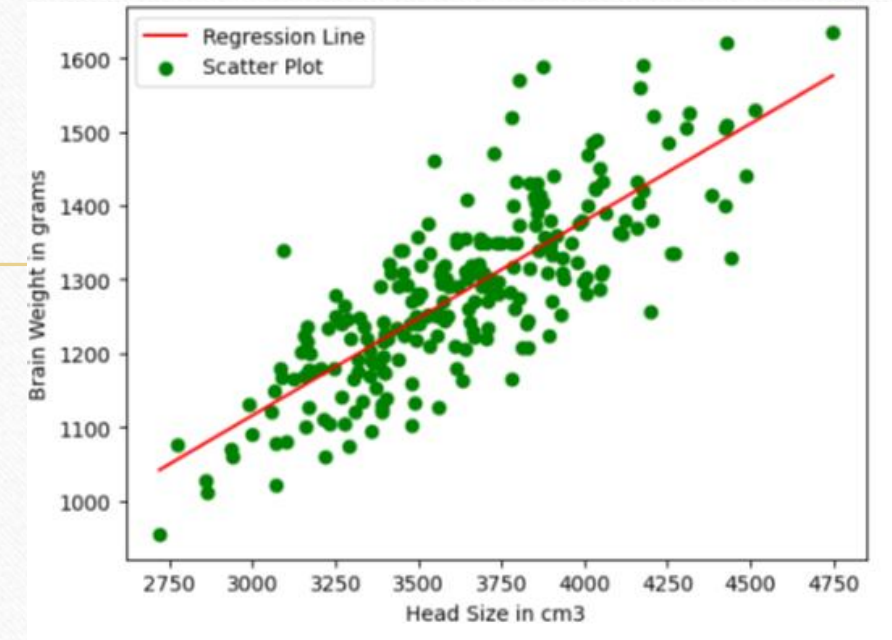
$$w_1 = \frac{N(\sum x_j y_j) - (\sum x_j)(\sum y_j)}{N(\sum x_j^2) - (\sum x_j)^2}; \quad w_0 = (\sum y_j - w_1(\sum x_j))/N$$

- `import numpy as np`
- `# Total number of values`
- `n = len(X)`
- `# calculating cross-deviation and deviation about x`
- `numer=n*np.sum(X*y)-np.sum(X)*np.sum(y)`
- `denom=n*np.sum(X*X)-(np.sum(X))**2`
- `w1=numer/denom`
- `# calculating regression coefficients`
- `w0=(np.sum(y)-w1*(np.sum(X)))/n`
- `w1,w0`

(0.2634293394893993, 325.57342104944274)

Plot Regression Line

- `import matplotlib.pyplot as plt`
- `max_x = np.max(X)`
- `min_x = np.min(X)`
- `# Calculating line values x and y`
- `x1 = np.linspace(min_x, max_x)`
- `y1 = w0 + w1 * x1`
- `# Plotting Line`
- `plt.plot(x1, y1, color='red', label='Regression Line')`
- `# Plotting Scatter Points`
- `plt.scatter(X, y, c='green', label='Scatter Plot')`
- `plt.xlabel('Head Size in cm3')`
- `plt.ylabel('Brain Weight in grams')`
- `plt.legend()`



Calculate RMSE

- `# Calculating Root Mean Squares Error`
- `rmse = 0`
- `for i in range(n):`
- `y_pred = w0 + w1 * X[i]`
- `rmse += (y[i] - y_pred) ** 2`
- `rmse = np.sqrt(rmse/n)`
- `print("RMSE=",rmse)`

$$\text{RMSE} = \sqrt{\sum_{i=1}^n \frac{1}{n} (\hat{y}_i - y_i)^2}$$

RMSE= 72.12062137837088

Calculating R2 Score

- `ss_tot = 0`
- `ss_res = 0`
- `y_mean=np.mean(y)`
- `for i in range(n):`
 - `y_pred = w0 + w1 * X[i]`
 - `ss_tot += (y[i] - y_mean) ** 2`
 - `ss_res += (y[i] - y_pred) ** 2`
- `r2 = 1 - (ss_res/ss_tot)`
- `print("R2 Score=",r2)`

$$R^2 = 1 - \frac{SS_{RES}}{SS_{TOT}} = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}$$

R2 Score= 0.6393117199570004

MULTIVARIATE LINEAR REGRESSION

- Real world problems rarely depend on just one input. Multivariate Linear Regression extends the same idea to many input variables at once

$$h_{sw}(\mathbf{x}_j) = w_0 + w_1x_{j,1} + \cdots + w_nx_{j,n} = w_0 + \sum_i w_ix_{j,i}.$$

MULTIVARIATE LINEAR REGRESSION

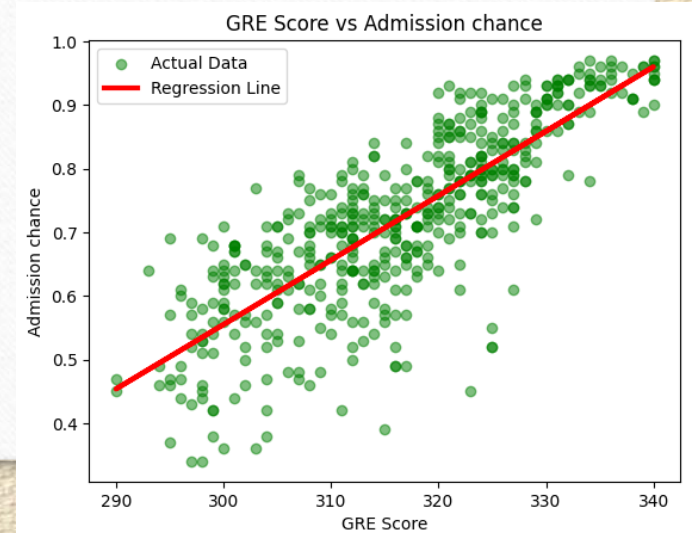
```
import pandas as pd
df=pd.read_csv("/content/Admission_Predict.csv")
columns=df.columns
df.drop("Serial#",axis=1,inplace=True)
y = df['Chance of Admit']
df.drop("Chance of Admit",axis=1,inplace=True)
df.head()
```

	Serial No.	GRE Score	TOEFL Score	University Rating	SOP	LOR	CGPA	Research
0	1	337	118	4	4.5	4.5	9.65	1
1	2	324	107	4	4.0	4.5	8.87	1
2	3	316	104	3	3.0	3.5	8.00	1
3	4	322	110	3	3.5	2.5	8.67	1
4	5	314	103	2	2.0	3.0	8.21	0

MULTIVARIATE LINEAR REGRESSION

- `import matplotlib.pyplot as plt`
- `from sklearn.linear_model import LinearRegression`
- `# Initialize and train the linear regression model`

- `simple_lr = LinearRegression()`
- `simple_lr.fit(df[['GRE Score']], y)`
- `plt.scatter(df['GRE Score'], y, color='green', label='Actual Data', alpha=0.5)`
- `plt.plot(df['GRE Score'], simple_lr.predict(df[['GRE Score']]), color='red', linewidth=3, label='Regression Line')`
- `plt.xlabel('GRE Score')`
- `plt.ylabel('Admission chance')`
- `plt.title('GRE Score vs Admission chance')`
- `plt.legend()`
- `plt.show()`



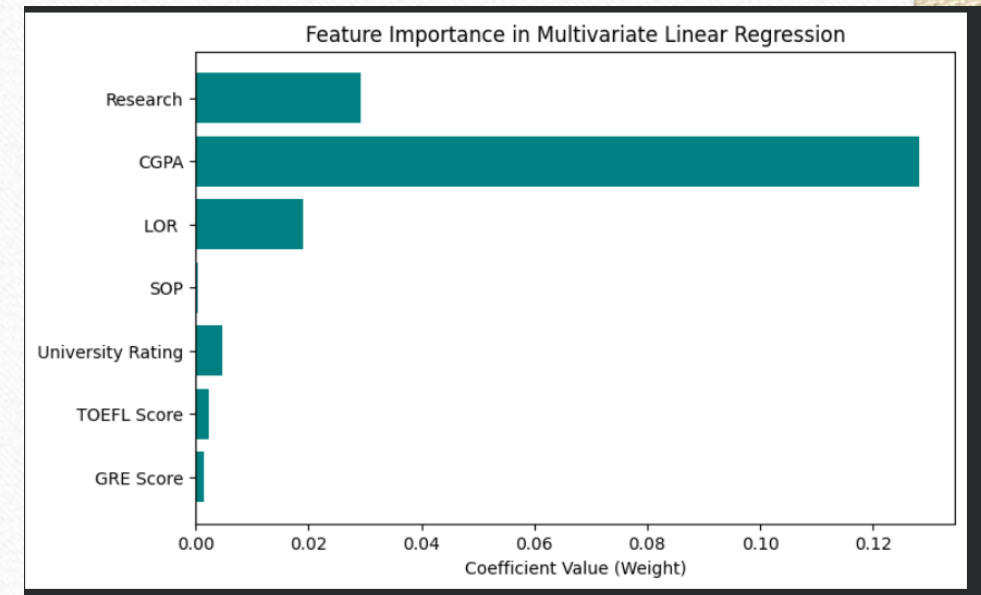
MULTIVARIATE LINEAR REGRESSION

```
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression

# Instantiate and train a multivariate linear regression model
lr = LinearRegression()
lr.fit(x_train, y_train)

coefficients = lr.coef_
features = df.columns

plt.figure(figsize=(8, 5))
plt.barh(features, coefficients, color='teal')
plt.xlabel('Coefficient Value (Weight)')
plt.title('Feature Importance in Multivariate Linear Regression')
plt.axvline(x=0, color='black', linewidth=0.8)
plt.show()
```



MULTIVARIATE LINEAR REGRESSION

- `from sklearn import metrics`
- `import numpy as np`
- `from sklearn.model_selection import train_test_split`

```
x_train, x_test, y_train, y_test = train_test_split(df, y, test_size=0.2)
```

- `rmse = np.sqrt(metrics.mean_squared_error(y_test, pred))`
- `print("RMSE:", rmse)`

```
RMSE: 0.18459205169520737
```

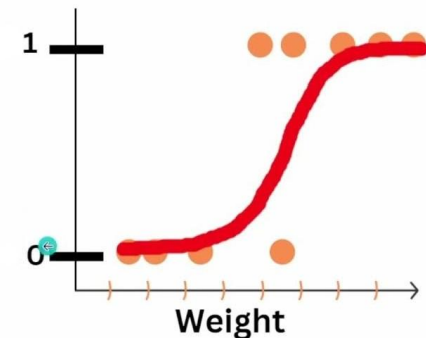
```
R2 Score: -1.078433002731218
```

Logistic Regression (Scratch)

- Logistic regression is a statistical and machine learning technique used for classification problems, where the goal is to predict a categorical outcome, most commonly a binary outcome (yes/no, 0/1, pass/fail), based on one or more independent variables
- linear regression predicts values with a straight line, while logistic regression predicts categories, and the output curve is not straight, it is S shaped, passes it through a special function called the sigmoid function, which converts any number into a probability range of 0-1.

Logistic Regression (Scratch)

- in linear relation we can draw straight line to classify but in this we can't.
 - The outcome is form of s.
-
- Backend working sigmoid or logit function
 - Sigmoid probability is equal to $1 / 1 + e^{-z}$. So e Euler's number = 2.7
 - Z is equal to linear equation equation $y = Mx + b$...
 - it converts input into 0 1 range...



Logistic Regression (Scratch)

- `import numpy as np`
- `import pandas as pd`

- `from sklearn.model_selection import train_test_split`
- `from sklearn.preprocessing import StandardScaler, LabelEncoder`
- `from sklearn.metrics import accuracy_score, classification_report, confusion_matrix`
- `# Load the dataset from a CSV file`
- `data = pd.read_csv('/content/Social_Network_Ads.csv')`

Logistic Regression (Scratch)

- # Preprocessing

- # Encode categorical variable 'Gender'

- labelencoder = LabelEncoder()

- data['Gender'] = labelencoder.fit_transform(data['Gender']) # Male=1, Female=0

- # Define features and target variable

- X = data[['Gender', 'Age', 'EstimatedSalary']].values # Select relevant columns

- y = data['Purchased'].values # Target variable

Logistic Regression (Scratch)

- # Split the dataset into training and testing sets
 - `X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)`
-

- # Scale the features

- `scaler = StandardScaler()`
- `X_train = scaler.fit_transform(X_train)`
- `X_test = scaler.transform(X_test)`
-

Logistic Regression (Scratch)

```
• # Logistic Regression Implementation
• class LogisticRegression:
•     def __init__(self, learning_rate=0.01, num_iterations=1000):
•         self.lr = learning_rate
•         self.iterations = num_iterations
•         self.weights = None
•         self.bias = None
•     def sigmoid(self, z):
•         return 1 / (1 + np.exp(-z))
•     def fit(self, X, y):
•         num_samples, num_features = X.shape
•         self.weights = np.zeros(num_features)
•         self.bias = 0
```

Logistic Regression (Scratch)

- for i in range(self.iterations):
 - `linear_model = np.dot(X, self.weights) + self.bias`
 - `y_predicted = self.sigmoid(linear_model)`
 - `# Update weights and bias by calculating Gradient`
 - `dw = (1 / num_samples) * np.dot(X.T, (y_predicted - y))`
 - `db = (1 / num_samples) * np.sum(y_predicted - y)`
 - `self.weights -= self.lr * dw`
 - `self.bias -= self.lr * db`
- `def predict(self, X):`
 - `linear_model = np.dot(X, self.weights) + self.bias`
 - `return [1 if i > 0.5 else 0 for i in self.sigmoid(linear_model)]`

$$dw = \frac{1}{n} \cdot X^T \cdot (y_{\text{predicted}} - y)$$

$$db = \frac{1}{n} \cdot \sum (y_{\text{predicted}} - y)$$

Logistic Regression (Scratch)

Train the Model

- `model = LogisticRegression(learning_rate=0.1, num_iterations=1000)`
- `model.fit(X_train, y_train)`

Evaluate the Model

- `predictions = model.predict(X_test)`
- `print("Accuracy:", accuracy_score(y_test, predictions))`
- `print("Confusion Matrix:\n", confusion_matrix(y_test, predictions))`
- `print("Classification Report:\n", classification_report(y_test, predictions))`

```
Accuracy: 0.8875
Confusion Matrix:
[[50  2]
 [ 7 21]]
Classification Report:
              precision    recall  f1-score   support

     0       0.88      0.96      0.92        52
     1       0.91      0.75      0.82        28

 accuracy          0.89          0.89          0.89          80
 macro avg         0.90          0.86          0.87          80
 weighted avg      0.89          0.89          0.88          80
```

Logistic Regression (Sklearn)

- `import pandas as pd`
- `from sklearn.model_selection import train_test_split`
- `from sklearn.preprocessing import StandardScaler, LabelEncoder`
- `from sklearn.linear_model import LogisticRegression`
- `from sklearn.metrics import accuracy_score, confusion_matrix, classification_report`
- `# Load dataset`
- `data = pd.read_csv("/content/Social_Network_Ads.csv")`
- `# Encode Gender (Male/Female → 1/0)`
- `le = LabelEncoder()`
- `data['Gender'] = le.fit_transform(data['Gender'])`

Logistic Regression (Sklearn)

- # Features & target
 - `X = data[['Gender', 'Age', 'EstimatedSalary']].values`
 - `y = data['Purchased'].values`
-
- # Train-test split
 - `X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)`
 - # Scale features
 - `scaler = StandardScaler()`
 - `X_train = scaler.fit_transform(X_train)`
 - `X_test = scaler.transform(X_test)`

Logistic Regression (Sklearn)

- `# Logistic Regression`
- `model = LogisticRegression()`
- `model.fit(X_train, y_train)`
- `# Predictions`
- `y_pred = model.predict(X_test)`
- `# Evaluation`
- `print("Accuracy:", accuracy_score(y_test, y_pred))`
- `print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))`
- `print("Classification Report:\n", classification_report(y_test, y_pred))`

Accuracy: 0.8875

Confusion Matrix:

[[50 2]

[7 21]]

Classification Report:

	precision	recall	f1-score	support
0	0.88	0.96	0.92	52
1	0.91	0.75	0.82	28
accuracy			0.89	80
macro avg	0.90	0.86	0.87	80
weighted avg	0.89	0.89	0.88	80

Polynomial regression

Polynomial regression fits a curved line instead of a straight line by adding powers of x (x^2 , x^3 , etc.) as extra features. It still uses the same linear regression method underneath, just applied to these transformed features, so it can capture non-linear patterns in data.

Polynomial Regression

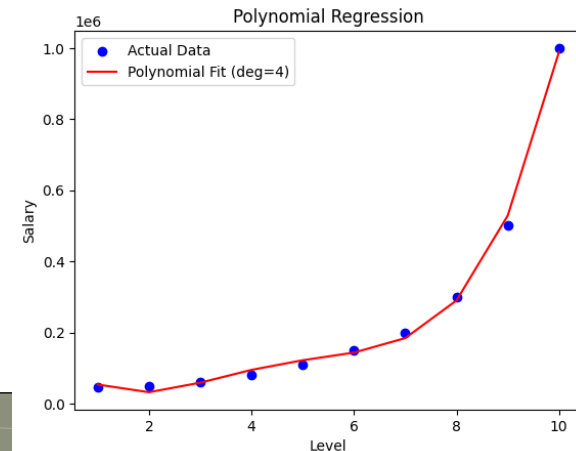
- *import pandas as pd*
- *import numpy as np*
- *import matplotlib.pyplot as plt*
- *from sklearn.linear_model import LinearRegression*
- *from sklearn.preprocessing import PolynomialFeatures*
- *# Load dataset*
- *data = pd.read_csv("Position_Salaries.csv")*
- *# Independent variable (Level) and dependent variable (Salary)*
- *X = data[["Level"]].values*
- *y = data["Salary"].values*

Polynomial Regression

- # Polynomial transformation (degree=4 for better curve fitting)
- `poly = PolynomialFeatures(degree=4)`
- `X_poly = poly.fit_transform(X)`
- # Train polynomial regression model
- `poly_model = LinearRegression()`
- `poly_model.fit(X_poly, y)`
- # Predictions
- `y_pred = poly_model.predict(X_poly)`

Draw the line....

- # Plot results
- `plt.scatter(X, y, color="blue", label="Actual Data")`
- `plt.plot(X, y_pred, color="red", label="Polynomial Fit (deg=4)")`
- `plt.xlabel("Level")`
- `plt.ylabel("Salary")`
- `plt.title("Polynomial Regression")`
- `plt.legend()`
- `plt.show()`
- Find out its R2 score and RMSE



Activity 1

Predict a person's **medical insurance cost** based on their personal details.

Dataset: Medical Cost Personal Dataset

Instructions

1. Load and explore the dataset.

1. Features include: age, bmi, children, smoker, region.
2. Target = charges (insurance cost).

2. Preprocess:

1. Apply **Encoding** to categorical features (region, smoker).
2. Scale numerical features.

3. Train a **Linear Regression model** to predict charges.

4. Evaluate using **RMSE & R^2 score**.

5. Plot **Predicted vs Actual Costs**.

Activity 2

Predict whether a customer will **churn** (leave the company) based on their behavior and usage patterns.

Dataset: Telco Customer Churn

Instructions:

1. Load dataset and preprocess categorical variables (Encoding).
2. Train a **Logistic Regression model** with features like tenure, MonthlyCharges, Contract, InternetService.
3. Evaluate using **Accuracy, Precision, Recall, F1-score**.
4. Plot **Confusion Matrix & ROC Curve**.

Activity 3

Predict **used car prices** based on mileage, age, and horsepower.

Dataset: Car Price Prediction

Instructions:

1. Choose features like `Kms_Driven`, `Year`, `Engine`, `Horsepower`.
2. Apply **Linear Regression** and record performance.
3. Apply **Polynomial Regression (degree=2,3,4)** and compare results.
4. Plot **fitted curves** for different polynomial degrees.